Materia: Programación IV

UNIDAD 9 · ESTADO GLOBAL CON ZUSTAND Y AUTENTICACIÓN

TRABAJO PRÁCTICO

Trabajo Práctico Integrador: estado global con Zustand y autenticación

Revisor de la unidad: Prof. Santiago Videla

Objetivos

Extender la aplicación full stack construida en la Unidad 8 (CRUD de categorías, productos e ingredientes) incorporando un módulo de seguridad. En el backend con FastAPI se implementan la autenticación con JWT en una cookie HttpOnly, la gestión de usuarios y la autorización por roles. En el frontend se consume la API autenticada con una instancia de Axios con interceptores, se mantiene la sesión con un store de Zustand y se protegen las rutas según el rol.

> **PROYECTO DE REFERENCIA**
>
> `seguridad_jwt_pg` (Actividad 2, versión corregida) implementa el módulo de usuarios, la cookie HttpOnly, CORS con credenciales, el `.env.example` y las dependencias `get_current_user`, `get_current_active_user` y `require_role`. Puede usarse como guía o como punto de partida. Su CRUD de categorías, en cambio, solo exige sesión y borra físicamente: a partir de él hay que aplicar `require_role(["admin"])` a las escrituras (punto c) e implementar el borrado lógico (punto e).

Marco teórico

Concepto	Aplicación en el proyecto	Actividad
JWT	Token firmado que acredita la sesión. El servidor lo emite en el login y lo valida en cada petición protegida.	A1 y A2
Cookie HttpOnly	Transporta el JWT. JavaScript no puede leerla, lo que impide robar el token mediante XSS.	A2
Roles y autorización (RBAC)	Cada endpoint de escritura exige el rol `admin` mediante `require_role`.	A2

Concepto	Aplicación en el proyecto	Actividad
`OAuth2PasswordBearer`	Esquema de FastAPI que extrae el token del encabezado `Authorization`. El proyecto lo extiende para leerlo de la cookie; la validación la hace la dependencia `get_current_user`.	A2
Axios	Cliente HTTP del frontend, con una instancia configurada (`baseURL`, `withCredentials`).	A5
Interceptores de Axios	Funciones que se ejecutan antes de cada petición y después de cada respuesta. El de respuesta maneja el 401 de forma global.	A5
AuthStore (Zustand)	Store global con los datos del usuario autenticado (nombre, email y rol) y las acciones de login y logout. No guarda el token.	A3 y A4
Middleware `persist`	Guarda en `sessionStorage` o `localStorage` los datos del usuario, para conservarlos al recargar la página.	A4
`ProtectedRoute`	Componente que redirige al login si no hay sesión, o a `/unauthorized` si el rol no está permitido.	— (se adelanta de la Unidad 10; React Router de unidades anteriores)

Consignas

Ejercicio 1 · Parte A – Backend: seguridad con FastAPI

50 puntos

Sobre el proyecto de la Unidad 8:

a) Módulo de usuarios. Crear `app/modules/usuarios/` con el modelo `Usuario` (`username`, `full_name`, `email`, `hashed_password`, `role` y `disabled`), los esquemas `UserCreate` y `UserPublic`, el repositorio, el servicio y el router, como en el proyecto de referencia.

b) Endpoints de autenticación (prefijo `/api/v1/auth`):

- `POST /register`: registra un usuario con rol `user`; la contraseña se guarda con hash bcrypt.
- `POST /token`: recibe `username` y `password` como formulario OAuth2, valida las credenciales y entrega el JWT en la cookie HttpOnly `access_token`.
- `POST /logout`: responde con una cabecera `Set-Cookie` que borra la cookie en el navegador.
- `GET /me`: devuelve los datos del usuario autenticado (requiere una sesión válida).

- `GET /admin/usuarios`, `POST /admin/usuarios/{id}/activar` y `POST /admin/usuarios/{id}/desactivar`: solo `admin`, como en el proyecto de referencia. Un usuario desactivado recibe 400 «Cuenta de usuario desactivada» en el login y en las rutas protegidas.

c) Protección de los endpoints existentes:

- Las operaciones de escritura (`POST`, `PUT`, `PATCH` y `DELETE`) sobre categorías, productos e ingredientes requieren el rol `admin`.
- Excepción: los pedidos (y las direcciones, si el proyecto las tiene) puede crearlos cualquier usuario autenticado con rol `user` o `admin`. El rol «CLIENTE» de la consigna original corresponde al rol `user` del curso.
- Las tablas de catálogo se cargan con un script `seed.py` y solo exponen `GET`: no tienen operaciones de escritura.
- Las operaciones de lectura (`GET`) están disponibles para cualquier usuario autenticado.
- El registro de usuarios es público (no requiere sesión).
- Se usan las dependencias reutilizables `get_current_user`, `get_current_active_user` y `require_role(roles)`.

d) CORS con credenciales, para que el navegador acepte y envíe la cookie desde el frontend:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

e) Borrado lógico en todos los modelos. Los registros no se eliminan de la base: `DELETE` marca el registro como inactivo (por ejemplo, con un campo `activo: bool`) y los listados devuelven solo los registros activos. En `Usuario`, el borrado lógico es el campo `disabled`, que se maneja con los endpoints de activar y desactivar.

f) Configuración y ejecución. La `SECRET_KEY` y la contraseña de la base se leen de un archivo `.env`, que no se entrega: se incluye un `.env.example` con marcadores. El backend se ejecuta con `uvicorn app.main:app --reload`.

Ejercicio 2 · Parte B – Frontend con React y TypeScript

50 puntos

g) Dependencias: `pnpm add axios zustand`.

h) Instancia de Axios en `src/api/axiosInstance.ts` (en la Actividad 5 el archivo se llama `client.ts`; el contenido es el mismo):

- `baseURL` apuntando al backend y `withCredentials: true`, para que el navegador envíe la cookie en cada petición.
- Interceptor de solicitud: no maneja el token. El token viaja en la cookie HttpOnly, que JavaScript no puede leer, y el navegador la adjunta solo.
- Interceptor de respuesta: si el backend responde 401 (salvo en el login y en `/auth/me`), limpia el AuthStore, de modo que la aplicación vuelve al login.

i) AuthStore en `src/store/authStore.ts`:

- Estado: el usuario autenticado (tipo `UsuarioAuth`, definido en `src/types/auth.types.ts`, con `username`, `full_name`, `email` y `role`) y un indicador de carga. **No guarda el token.**
- Acciones: `login(username, password)`, que llama a `POST /auth/token` y luego a `GET /auth/me`; `logout()`, que llama a `POST /auth/logout` y limpia el store; y `checkSession()`, que confirma la sesión con `GET /auth/me` al iniciar la aplicación.
- Middleware `persist` con `partialize`, para guardar solo los datos del usuario.
- El store se exporta como el hook `useAuthStore` y lo consumen `Navbar`, `ProtectedRoute` y `LoginPage`.

j) Páginas de autenticación:

- `LoginPage.tsx`: formulario con TanStack Form que llama a `login` del AuthStore. Si las credenciales son incorrectas, muestra el mensaje sin recargar la página.
- `RegisterPage.tsx`: formulario de registro con TanStack Form; al completarse, redirige al login.
- `UnauthorizedPage.tsx`: página simple de acceso denegado.

k) `ProtectedRoute` en `src/components/ProtectedRoute.tsx`:

- Lee el AuthStore y recibe opcionalmente los roles permitidos (`allowedRoles`).
- Mientras se confirma la sesión, muestra un indicador de carga.
- Sin sesión, redirige a `/login`; con sesión pero rol no permitido, a `/unauthorized`.
- Patrón mínimo de la redirección: `if (!user) return <Navigate to="/login" replace />;` (React Router). La Unidad 10 desarrolla este componente en profundidad.

l) Rutas en `App.tsx`:

Ruta	Página	Acceso
`/login`, `/register`, `/unauthorized`	Login, registro y acceso denegado	Públicas
`/categories`, `/products`, `/ingredients`	Listados del CRUD	Cualquier usuario autenticado

Ruta	Página	Acceso
`/admin/users`	Listado de usuarios con los botones Activar y Desactivar	Solo `admin`

m) Interfaz condicional por rol:

- `Navbar.tsx` muestra el nombre y el rol del usuario y un botón «Cerrar sesión».
- En las páginas de listado, los botones de crear, editar y eliminar se muestran solo si el rol es `admin`. Ocultarlos no reemplaza la validación del backend, que responde 403 a un usuario `user`.

Estructura de carpetas esperada

```
src/
├── api/
│   └── axiosInstance.ts
├── components/
│   ├── Navbar.tsx
│   └── ProtectedRoute.tsx
├── pages/
│   ├── LoginPage.tsx
│   ├── RegisterPage.tsx
│   ├── CategoriesPage.tsx
│   ├── ProductsPage.tsx
│   ├── IngredientsPage.tsx
│   ├── AdminUsersPage.tsx
│   └── UnauthorizedPage.tsx
├── store/
│   └── authStore.ts
├── types/
│   └── auth.types.ts
└── App.tsx
```

Comportamiento esperado

- La aplicación inicia en `/login` si no hay sesión activa.
- Un usuario puede registrarse y luego iniciar sesión.
- Al iniciar sesión, el navegador guarda la cookie HttpOnly y el AuthStore guarda los datos del usuario.
- Al recargar la página, la sesión se confirma con `/auth/me` sin volver a pedir las credenciales.
- Un usuario con rol `user` ve los listados, pero no puede crear, editar ni eliminar.
- Un usuario con rol `admin` tiene acceso completo al CRUD y a `/admin/users`.

- Al cerrar sesión, se borra la cookie, se limpia el AuthStore y se vuelve al login.
- Si la sesión vence, el interceptor de Axios detecta el 401 y la aplicación vuelve al login.

Historias de usuario

Las siguientes historias de usuario describen, desde la perspectiva de quien opera el sistema, el comportamiento esperado de la aplicación. Cada historia sigue el formato «Como... quiero... para...» y se acompaña de sus criterios de aceptación, que constituyen la condición verificable de que la historia se implementó correctamente.

ID	Historia de usuario y criterios de aceptación	Verificación
HU-01	**Registro de usuario** Como persona sin cuenta, quiero registrarme para acceder al sistema. *Criterios de aceptación:* • `POST /auth/register` recibe `username`, `full_name`, `email` y `password`, y guarda la contraseña con hash bcrypt. • Responde 201 con los datos del usuario (sin la contraseña), 409 si el usuario o el email ya existen y 422 si los datos son inválidos. • El frontend redirige a `/login` al completar el registro.	Frontend y backend
HU-02	**Inicio de sesión** Como usuario registrado, quiero iniciar sesión para acceder a las funciones protegidas. *Criterios de aceptación:* • `POST /auth/token` valida las credenciales y envía el JWT en una cookie HttpOnly. • El frontend obtiene el usuario con `GET /auth/me` y guarda nombre, email y rol en el AuthStore. • Tras el login se redirige a `/categories`. • Con credenciales inválidas se muestra un mensaje de error sin recargar la página.	Frontend y backend
HU-03	**Cierre de sesión** Como usuario autenticado, quiero cerrar la sesión para proteger mis datos. *Criterios de aceptación:* • `POST /auth/logout` responde con una cabecera `Set-Cookie` que borra la cookie en el navegador. • El AuthStore se limpia y se redirige a `/login`. • El botón «Cerrar sesión» se ve en el `Navbar` cuando hay sesión activa.	Frontend y backend

ID	Historia de usuario y criterios de aceptación	Verificación
HU-04	**Sesión al recargar y perfil** Como usuario autenticado, quiero que la sesión se mantenga al recargar la página y ver mis datos en el `Navbar`. *Criterios de aceptación:* • Al iniciar la aplicación se llama a `GET /auth/me`; mientras tanto, se muestra un indicador de carga. • El `Navbar` muestra el nombre y el rol del usuario. • Si la sesión venció, el interceptor de Axios detecta el 401, limpia el AuthStore y la aplicación vuelve a `/login`.	Frontend
HU-05	**Acceso condicional por rol** Como usuario con rol `user`, quiero ver los listados sin poder modificarlos; como `admin`, quiero gestionarlos. *Criterios de aceptación:* • Los botones de crear, editar y eliminar solo se ven si el rol es `admin`. • Un usuario `user` que envía `POST`, `PUT`, `PATCH` o `DELETE` recibe 403 del backend. • Los endpoints de escritura exigen el rol `admin` mediante `require_role`.	Frontend y backend
HU-06	**Protección de rutas** Como persona sin sesión, quiero que la aplicación me lleve al login si intento entrar a una ruta protegida. *Criterios de aceptación:* • `ProtectedRoute` lee el AuthStore; sin sesión, redirige a `/login`. • `/categories`, `/products` e `/ingredients` requieren sesión; `/admin/users`, además, el rol `admin`. • Un usuario `user` que entra a `/admin/users` es redirigido a `/unauthorized`. • `/login` y `/register` son públicas.	Frontend
HU-07	**Instancia e interceptores de Axios** Como usuario autenticado, quiero que la aplicación maneje la sesión vencida de forma automática. *Criterios de aceptación:* • Todas las peticiones usan la instancia de `axiosInstance.ts`, con `withCredentials: true`. • El interceptor de respuesta detecta el 401 y limpia la sesión. • Ningún archivo del frontend lee ni guarda el token.	Frontend

Reglas de negocio y de seguridad

ID	Regla	Entidad / verificación
RN-01	La contraseña se almacena con hash bcrypt, nunca en texto plano.	Usuario / el campo contiene un hash bcrypt
RN-02	El nombre de usuario y el email son únicos.	Usuario / 409 Conflict
RN-03	El JWT se transmite exclusivamente en la cookie HttpOnly.	Auth / el frontend no accede al token
RN-04	Los endpoints de escritura requieren el rol `admin`.	Categoría, Producto, Ingrediente / 403 Forbidden
RN-05	Los pedidos (y las direcciones, si existen) pueden crearlos los usuarios con rol `user`.	Pedido, Dirección / 201
RN-06	Las tablas de catálogo se cargan con `seed.py` y solo exponen `GET`.	Catálogos / 405 ante escrituras
RN-07	Los endpoints de lectura están disponibles para cualquier usuario autenticado.	Categoría, Producto, Ingrediente / 200
RN-08	El registro de usuarios es público y asigna siempre el rol `user`.	Auth / 201
RN-09	Un email inválido o una contraseña de menos de 8 caracteres se rechazan.	Usuario / 422
RN-10	El JWT se valida en cada petición protegida mediante `get_current_user`.	Auth / 401 si no es válido
RN-11	Los registros no se eliminan físicamente: se aplica borrado lógico en todos los modelos (en `Usuario`, con `disabled`).	Todos / el registro sigue en la base
RN-12	Toda petición inválida responde con un cuerpo `{"detail": ...}` (cadena en 400, 404 y 409; lista de errores de validación en 422) sin modificar datos.	Todas / 400, 404, 409, 422
RN-13	El AuthStore con `persist` conserva los datos del usuario entre recargas; la sesión se confirma con `/auth/me`.	Frontend / Zustand `persist`

Criterios de evaluación

Criterio	Descripción	Puntos
Autenticación (backend)	Registro, login, logout y `/me` funcionando con persistencia en la base.	15

Criterio	Descripción	Puntos
JWT y cookie HttpOnly	Token generado y firmado, enviado en la cookie HttpOnly y validado en las rutas protegidas; CORS con credenciales.	15
Autorización por roles	`require_role` implementado; escritura solo para `admin`, salvo la creación de pedidos por usuarios `user`; catálogos de solo lectura cargados con `seed.py`.	15
Borrado lógico	`DELETE` marca los registros como inactivos en todos los modelos y los listados los excluyen; en `Usuario`, mediante `disabled`.	5
Axios e interceptores	Instancia con `withCredentials`; interceptor de respuesta que maneja el 401.	10
AuthStore (Zustand y `persist`)	Store exportado como `useAuthStore`, con login, logout y `checkSession`; persiste solo los datos del usuario, sin token.	15
`ProtectedRoute` y navegación	Rutas protegidas por sesión y por rol; indicador de carga; redirecciones correctas; página `/admin/users` con Activar y Desactivar; formularios de login y registro con TanStack Form.	10
Interfaz condicional por rol	Botones de edición solo para `admin`; `Navbar` con nombre, rol y cierre de sesión.	10
Tipado con TypeScript	Interfaces para `UsuarioAuth`, el login y las respuestas del backend, sin `any` implícito.	5
TOTAL		100

Formato de entrega

- Dos archivos .zip: uno con el proyecto frontend y otro con el proyecto backend.
- Antes de comprimir, eliminar `node_modules` (frontend) y `.venv` (backend).
- No incluir el archivo `.env`: entregar un `.env.example` con marcadores.

Recursos adicionales

Documentación oficial:

- React: https://react.dev/
- TypeScript: https://www.typescriptlang.org/docs/
- Tailwind CSS: https://tailwindcss.com/docs/

- Vite: https://vite.dev/
- FastAPI: https://fastapi.tiangolo.com/